Written Memo — Constrained PPE Detection & Reasoning API

1. Why this domain and dataset

I chose construction-site PPE (personal protective equipment) compliance detection, using the Construction Site Safety Image Dataset from Roboflow Universe (mirrored on Kaggle by snehilsanyal) -- 2,801 images across 10 classes: Hardhat, Mask, NO-Hardhat, NO-Mask, NO-Safety Vest, Person, Safety Cone, Safety Vest, machinery, vehicle.

Two reasons for this choice. First, at least three classes -- NO-Hardhat, NO-Mask, NO-Safety Vest -- encode the *absence* of PPE. These are not native categories in COCO-pretrained RT-DETR's detection classes, so fine-tuning on this custom dataset was required rather than optional. Second, it has an obvious real-world use: flagging missing safety gear on a job site in near real time.

The dataset came pre-annotated from Roboflow; I did not add new labels. I created my own deterministic train/validation/test split so that the evaluation protocol was under my control and reproducible, rather than using the split that shipped with the dataset (details below).

2. My split strategy

Because each image may contain multiple object classes, I used a custom deterministic split that prioritizes the rarest class present in each image when assigning train/val/test, rather than a standard multi-label stratification algorithm (see `src/split_dataset.py`). Seed 42, 75/15/10 proportions.

Reasoning: a plain random split can starve a rare class like Safety Cone out of validation/test, making its metrics unreliable. I used 15% for validation (not the more typical 10%) because with only ~2,800 images across 10 classes, a smaller val set risks too few rare-class examples for a stable per-class mAP during training.

Actual result: 2,101 train / 422 validation / 278 test images.

3. What my evaluation actually showed

Held-out test set (278 images, never seen in training or validation):

• Overall mAP50-95: **0.6624** | mAP50: **0.9048**

• Precision (mean): 0.9184 | Recall (mean): 0.8652

Class	AP50	Precision	Recall
Hardhat	0.860	0.906	0.803
Mask	0.939	0.932	0.919
NO-Hardhat	0.905	0.913	0.897
NO-Mask	0.893	0.910	0.874
NO-Safety Vest	0.939	0.940	0.903
Person	0.956	0.952	0.912
Safety Cone	**0.764**	0.878	**0.637**
Safety Vest	0.937	0.937	0.906
machinery	0.973	0.950	0.946
vehicle	0.882	0.868	0.855

RT-DETR-l, fine-tuned from COCO-pretrained weights, 100 epochs, 640px, batch 16, single Tesla T4 (Colab), 251.2 minutes. I chose RT-DETR-l over the larger RT-DETR-x for a better accuracy/training-time balance given the T4's VRAM and my timeline.

Safety Cone is clearly my weakest class by AP50 (0.764) and recall (0.637). Section 4 breaks down exactly what's driving this.

A methodology note: the confusion matrix and the reported per-class recall should not be interpreted as the same statistic. The confusion matrix is a threshold-specific view of prediction/ground-truth matching, while the reported recall comes from the detector's evaluation procedure integrating across the precision-recall curve. I use the evaluation metrics table for quantitative model comparison and the confusion matrix primarily for qualitative error analysis.

What these numbers don't tell me: how the model performs on RAP's hidden eval set (which may differ in lighting/angle/site type), or whether a specific violation is attributed to a specific worker -- my design doesn't do that (Section 5).

4. Five things my model gets wrong, and why

Pulled from the confusion matrix on my test set (`results/confusion_matrix.png`). I used it to inspect class-level false positives and missed detections: the background-related cells represent predictions or ground-truth instances that were not matched to the corresponding class.

1. Safety Cone's main problem is false-positive predictions. Per the confusion matrix, 348 cones were correctly detected, 69 were missed, and 196 false-positive Cone predictions were recorded. I noticed that these cell counts do not exactly match the raw label-file count for this class. However, the confusion matrix and raw annotation counts are not directly comparable because they are produced through different matching and evaluation procedures. I therefore do not treat the confusion-matrix cell counts as exact ground-truth instance totals. The reliable conclusion is that Safety Cone is my weakest class by recall and shows substantial false-positive behavior, consistent with visually similar site objects such as warning signage or stacked materials being confused with cones.

2. Person has the highest false-positive count of any class (201). Person is one of my strongest classes by AP50 and precision (910 correctly detected, only 49 missed), but the model produces false- positive Person predictions 201 times -- more than any other class. Likely cause: partial human-like shapes (reflections, heavily occluded limbs, people partially hidden behind machinery) getting flagged as full detections.

3. Person also has real missed detections in busy scenes (49). Given how well-represented and otherwise accurate this class is, these misses are more likely occlusion in crowded site photos than a data scarcity issue.

4. Safety Vest shows the same false-positive/miss imbalance at a smaller scale. 71 false-positive Safety Vest predictions vs. 18 misses -- consistent with bright, simple, PPE-adjacent objects being over-triggered on across multiple classes, not just cones.

5. Small/occluded PPE is a recurring detection weakness across classes, not just Safety Cone. Hardhat has the second-lowest recall of any class (0.803), noticeably behind Mask (0.919), Person (0.912), and machinery (0.946). Combined with Safety Cone's weakness, this points to a consistent pattern: smaller PPE items that occupy few pixels or get partially occluded by the worker's body or nearby equipment are harder for the model to detect reliably, regardless of which specific class they belong to. Likely causes: limited effective resolution for small objects at 640px input size, and occlusion in busy site photos. A likely improvement would be higher-resolution training/inference or more examples of small, partially occluded PPE.

5. How my reasoning layer decides when to call the detector

Three plain Python functions, no framework (`api/reasoning.py`):

1. `needs_detection(question)` -- keyword/regex check for whether a question is about the image at all. 2. `reason_over_detections(question, detections)` -- handles counting, PPE-violation questions (including the brief's own example, "is anyone not wearing a helmet?"), and most-common-object questions. It uses violation classes like NO-Hardhat/NO-Mask/NO-Safety Vest as *image-level* evidence only -- it does not establish person-to-PPE ownership, so it never claims a specific person is non-compliant. 3. A confidence guardrail: it rejects an answer when no relevant detection exceeds a 0.35 confidence threshold. For people-dependent questions, it additionally requires at least one sufficiently confident Person detection before answering.

Real example, captured live from my running API:

```
Question: "Is anyone not wearing a safety vest?"
Image: airport_inside_0073_...jpg (a non-construction negative example)
Response: {
  "used_detector": true,
  "raw_detections": [
    {"class": "machinery", "confidence": 0.95, "box": [...]},
    {"class": "machinery", "confidence": 0.91, "box": [...]}
  ],
  "answer": "I don't have enough confident detection evidence to answer
            this reliably. No sufficiently confident Person detection
            was available.",
  "confident": false
}
```

No sufficiently confident Person detection was available, so the system correctly said it couldn't answer a people-focused question, rather than guessing from unrelated detections.

6. What's still weak, and what I'd fix with more time

• **No person-to-PPE attribution.** I can say "a NO-Safety-Vest box exists," not "worker #2 lacks a vest." If asked directly: the detector produces separate Person and NO-Safety-Vest boxes, but I haven't implemented person-to-PPE association, so my reasoning layer treats violations as image-level evidence rather than attributing them to a specific person. I would explore person-to-PPE association using spatial matching, such as IoU or containment-based rules, followed by validation on annotated examples.

• **A real bug I found and fixed by testing my own API.** During API testing, I discovered that the natural-language term "helmet" was not mapped to the dataset's "Hardhat" class, causing incorrect violation routing. I added an explicit PPE synonym mapping (`PPE_SYNONYMS` in `api/reasoning.py`) and verified the corrected behavior with live API tests. This also exposed a broader limitation: keyword-based routing may still fail on unseen phrasing.

• **Safety Cone's dominant issue is false positives, not misses** (see Section 4) -- more time would go toward reducing confusion with visually similar site clutter, not just adding more training images.

• **My 0.35 confidence threshold is a reasonable default, not tuned** against a precision/recall trade-off for this task specifically.